

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

MediClinic

Réalisé par : AKKAR Marwa
CHORFI Assiya

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM
Année : 2025/2026

Remerciements

Nous tenons à exprimer nos sincères remerciements à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous remercions tout particulièrement notre encadrante, Mme Douae EL HILA, pour son accompagnement, ses précieux conseils et sa disponibilité tout au long de ce travail.

Nous remercions également l'ensemble des enseignants de la filière Génie Informatique pour la qualité de leur enseignement.

Enfin, nous remercions nos familles et amis pour leur soutien indéfectible.

Table des matières

Remerciements	1
Introduction	6
1 Présentation du projet	7
1.1 Description de l'application.....	7
1.2 Besoins fonctionnels.....	7
1.3 Besoins non fonctionnels	7
2 Conception	8
2.1 Architecture MVC.....	8
2.2 Diagramme de classes UML	8
2.3 Diagramme de cas d'utilisation	8
2.4 Diagramme de séquence (optionnel)	8
3 Environnement de travail	9
3.1 Outils et technologies utilisés.....	9
3.2 Structure du projet.....	9
4 Répartition des tâches	10
4.1 Tâches de l'étudiant 1 –AKKAR Marwa	10
4.2 Tâches de l'étudiant 2 –CHORFI Assiya.....	10
5 Explication du code	11
5.1 Connexion à la base de données – Database.java	11
5.2 Les classes modèles	11
5.3 Les classes DAO.....	11
5.4 Les contrôleurs.....	11
5.5 Les fichiers FXML	11
5.6 Fonctionnalité spécifique au projet.....	11
6 Interfaces de l'application	12
6.1 Fenêtre principale	12
6.2 Interfaces de gestion de l'entité 1.....	12
6.3 Interfaces de gestion de l'entité 2.....	12
6.4 Tableau de bord – Statistiques	12
6.5 Menus et dialogues.....	12
6.6 Export de données.....	12
Conclusion	13

Table des figures

Liste des tableaux

4.1	Tâches réalisées par l'étudiant 1	10
4.2	Tâches réalisées par l'étudiant 2	10

Introduction

L'objectif de ce mini-projet est de concevoir et développer une application de gestion de clinique médicale nommée « MédiClinic ».

Cette application a pour but de faciliter la gestion des patients, des rendez-vous, des consultations, des dossiers médicaux et de la facturation au sein d'un cabinet médical ou d'une clinique.

Dans le cadre du module « Développement Java IHM », nous avons mis en œuvre une interface graphique en JavaFX, une architecture MVC et une base de données relationnelle MySQL. Ce rapport détaille les différentes phases de notre projet, de l'analyse des besoins à la conception et à l'implémentation, en passant par les choix technologiques et la répartition des tâches.

Chapitre 1

Présentation du projet

1.1 Description de l'application

L'application MédiClinic est un système de gestion intégré destiné aux cliniques et cabinets médicaux. Elle permet :

- La gestion des patients (création, modification, recherche).
- La gestion des rendez-vous (prise, modification, annulation).
- La gestion des consultations (saisie des constantes, diagnostic, ordonnance).
- La gestion des dossiers médicaux partagés (DMP).
- La génération de factures et de quittances.
- L'export de documents (ordonnances, rapports).

1.2 Besoins fonctionnels

Les besoins fonctionnels identifiés sont les suivants :

- Authentification sécurisée des utilisateurs (réceptionniste, médecin, administrateur).
- Gestion complète des patients (CRUD).
- Gestion des consultations : enregistrement des constantes, diagnostic, prescriptions.
- Génération automatique de factures selon les tarifs et la mutuelle.
- Recherche et filtrage des patients et rendez-vous.
- Export des ordonnances au format PDF

1.3 Besoins non fonctionnels

Les besoins non fonctionnels pris en compte sont :

- Interface utilisateur ergonomique et intuitive.
- Réactivité et performance de l'application.
- Sécurité des données (accès contrôlé, confidentialité).
- Modularité et extensibilité du code.
- Compatibilité avec les systèmes d'exploitation courants (Windows, Linux, macOS). Sauvegarde automatique et intégrité des données.

Chapitre 2

Conception

2.1 Architecture MVC

Nous avons adopté l'architecture Modèle-Vue-Contrôleur (MVC) pour séparer clairement les responsabilités :

- **Modèle** : classes représentant les données (Patient, RendezVous, Consultation, etc.) situées dans le package `com.clinic.model`, et les classes d'accès aux données (repository) pour l'interaction avec la base.
- **Vue** : interfaces utilisateur définies en FXML (JavaFX) dans le dossier `resources/fxml` et leurs contrôleurs associés.
- **Contrôleur** : logique métier, gestion des événements et coordination entre le modèle et la vue ; ces classes se trouvent dans `com.clinic.controller`. Cette architecture facilite la maintenance, les tests et l'évolution de l'application.

2.2 Diagramme de classes UML

Le diagramme de classes illustre les principales entités et leurs relations. Les classes principales sont :

Patient : id, nom, prénom, dateNaissance, téléphone, email, mutuelle, etc.

Medecin : id, nom, prénom, spécialité, etc.

RendezVous : id, dateHeure, statut, patient, médecin.

Consultation : id, date, constantes (tension, pouls, température), diagnostic, traitement, ordonnance.

Facture : id, montant, date, modePaiement, consultation.

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

Les outils et technologies employés sont :

- JavaFX : pour l'interface graphique.
- MySQL : système de gestion de base de données relationnelle.
- JDBC : pour la connexion à la base de données.
- Scene Builder : pour la conception des interfaces FXML.
- IntelliJ IDEA / Eclipse : environnement de développement intégré.
- Git : pour le contrôle de version.
- Maven : gestionnaire de dépendances et outil de construction (fichier pom.xml).
- PlantUML : pour la modélisation UML.
- iTextPDF : pour la génération de documents PDF.

3.2 Structure du projet

L'organisation du projet suit une structure Maven standard. Voici l'arborescence principale :

```
src/  
  main/  
    java/  
      com.clinic/  
        config/          # Classes de configuration (connexion DB, etc.)  
        controller/      # Contrôleurs JavaFX  
        model/           # Classes métier (entités)  
        repository/      # DAO : accès aux données  
        service/         # Logique métier et services  
          Launcher       # Point d'entrée  
          MainApp        # Classe principale JavaFX  
      resources/  
        fxml/            # Fichiers FXML (vues)  
        styles/          # Feuilles de style CSS  
  target/                # Fichiers compilés  
  pom.xml                # Configuration Maven  
  ...
```

Chapitre 4

Répartition des tâches

4.1 Tâches de l'étudiant 1 – AKKAR Marwa

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de l'architecture du projet JavaFX	Structure globale du projet	TERMINE
Développement des modèles métier	Models/*	TERMINE
Implémentation de la couche DAO et accès aux données	Dao/*	TERMINE
Configuration de la connexion à la base de données	Config/*	TERMINE
Création du point d'entrée de l'application	MainApp.java	TERMINE

TABLE 4.1 – Tâches réalisées par l'étudiant 1

4.2 Tâches de l'étudiant 2 – CHORFI Assiya

Tâche réalisée	Fichier(s) concerné(s)	Statut
Réalisation du diagrammes	Uml/*	TERMINE
Développement des contrôleurs JavaFX	controllers/*	TERMINE
Conception des interfaces utilisateur	resources/* (FXML, CSS, etc.)	TERMINE
Développement des classes utilitaires	utils/*	TERMINE

TABLE 4.2 – Tâches réalisées par l'étudiant 2

Chapitre 5

Explication du code

5.1 Connexion à la base de données

La classe `DatabaseConfig` (dans le package `config`) gère la connexion à MySQL via JDBC. Elle utilise le pattern Singleton pour assurer une connexion unique.

Listing 6.1 – Exemple de connexion avec Singleton

```
public class DatabaseConfig {
    private static Connection connection;
    private static final String URL = "jdbc:mysql://localhost:3306/medicli";
    private static final String USER = "root";
    private static final String PWD = "password";

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            connection = DriverManager.getConnection(URL, USER, PWD);
        }
        return connection;
    }
}
```

5.2 Les classes modèles

Les classes métier (`Patient`, `RendezVous`, etc.) sont placées dans `com.clinic.model`. Elles contiennent les attributs, les constructeurs, les getters/setters et éventuellement des méthodes utilitaires.

5.3 Les classes DAO

Chaque repository (ex. `PatientRepository`) implémente les opérations CRUD pour une entité donnée, en utilisant `DatabaseConfig`.

5.4 Les contrôleurs

Les contrôleurs JavaFX (package `controller`) gèrent les événements de l'interface (clics, saisies) et interagissent avec les services.

5.5 Fonctionnalité spécifique au projet

Une fonctionnalité clé est la génération d'ordonnances numériques. Le service `PDFGenerator` utilise `iTextPDF` pour créer un fichier PDF contenant les prescriptions du médecin.

Chapitre 6

Interfaces de l'application

6.1 Fenêtre principale

MediClinic - Authentification

Connexion

Nom d'utilisateur

Username

Mot de passe

Se connecter

6.2 Interfaces de gestion de l'entité 1

MediClinic - Réception

Gestion des Patients

Rechercher par Nom ou CIN... Rechercher

+ Nouveau Patient Prendre RDV

Nom	Prénom	CIN	Téléphone	Mutuelle
akkar	marwa	f11111	0697741189	RAMED
chorfi	assiya	f2342	0697741189	CNSS
ali	akkar	f111	087587	CNSS
safae	akkar	g5657	74328056	Assurance Privée
fadwa	akkar	g78	873149	Aucune
anas	bb	g865	87348	Aucune
najat	akkar	j6799	0697741189	CNOPS
marwa	akkar	f1113	0797741189	CNSS

File d'attente / Mouvements du jour

Patient	Heure	Statut	Ordonnance	Facture / Paiement
akkar marwa	09:30	TERMINE	Voir PDF	Facture / \$
anas bb	10:00	TERMINE	Voir PDF	Facture / \$
ali akkar	10:30	EN_ATTENTE	Voir PDF	Facture / \$

Chapitre 6

6.3 Interfaces de gestion de l'entité 2

[illegible]

6.4 Menus et dialogues

Consultation en cours

Consultation : ali akkar

Tension (ex: 12/8)

Poids (kg)

Diagnostic :

Observations :

Terminer et Générer Ordonnance

Chapitre 6

6.5 Export de données

MEDICLINIC

Centre Médical Pluridisciplinaire

14 Boulevard des Pyramides

Casablanca, Maroc

Tel: +212 5 22 10 20 30

Email: contact@mediclinic.ma

Dr Olivier D. Dr Bernard A. Dr Priscilla R. Dr Peter T.

ORDONNANCE

Patient : ANAS bb

Fait le : 2026-06-22

PRESCRIPTION :

1. Prendre Doliprane 1g : 1 comprimé 3 fois par jour (matin, midi et soir) pendant 3 jours. À prendre au moment des repas.

Cachet et Signature

Conclusion

Ce mini-projet nous a permis de mettre en pratique les concepts du développement d'applications Java avec interface graphique en JavaFX. Nous avons réussi à réaliser une application fonctionnelle de gestion de clinique, répondant aux besoins fonctionnels et non fonctionnels définis en amont. Les difficultés rencontrées (gestion des dates, validation des formulaires, génération PDF) ont été surmontées grâce à une bonne documentation et un travail d'équipe. Les perspectives d'amélioration incluent l'ajout d'une gestion des utilisateurs avec rôles, une synchronisation avec un calendrier externe, et une version mobile. Ce projet a renforcé nos compétences en conception orientée objet, en modélisation UML, en programmation événementielle et en intégration de bases de données.

Références

Documentation officielle JavaFX : <https://openjfx.io>

Documentation MySQL : <https://dev.mysql.com/doc/>

Oracle Java JDBC : <https://docs.oracle.com/javase/tutorial/jdbc/>

iTextPDF : <https://itextpdf.com/>

PlantUML : <https://plantuml.com/>

Apache Maven : <https://maven.apache.org/>